



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Jagoda WIĘCEK

Nr albumu: 300503

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

Dr inż. Ewa Lach

KATEDRA Grafiki, Wizji Komputerowej i Systemów Cyfrowych

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2024

Tytuł pracy

Gra planszowa 2.5D z uczeniem maszynowym komputerowych graczy

Streszczenie

(Streszczenie pracy.)

Słowa kluczowe

Model, agent, unity, uczenie maszynowe

Thesis title

2.5D board game with machine learning for computer players

Abstract

(Thesis abstract.)

Key words

Model, agent, unity, machine learning

Spis treści

1	Wstęp	1
1.1	Cel pracy	1
1.2	Zakres pracy	1
1.3	Charakterystyka rozdziałów	1
2	Analiza tematu	3
2.1	Sformułowanie problemu	3
2.2	Znane rozwiązania	4
3	Wymagania i narzędzia	5
3.1	Wymagania funkcjonalne	5
3.2	Wymagania нефункционалне	6
3.3	Opis narzędzi	7
3.3.1	Unity	7
3.3.2	C#	7
3.3.3	Python	7
3.3.4	ML-Agents	8
3.4	Przypadki użycia	9
4	Specyfikacja zewnętrzna	11
4.1	Wymagania sprzętowe i programowe	11
4.2	Instalacja	11
4.2.1	unity	11
4.2.2	biblioteka ml-agents	11
4.3	Aktywacja	11
4.4	Sposób obsługi	11
4.5	Ekran aplikacji	11
4.5.1	Zasady gry	11
4.5.2	Opis ekranu gry	11

5	Specyfikacja wewnętrzna	13
5.1	Biblioteka ml-agents	13
5.2	Klasa PlayerIA	13
5.3	Klasa GameController	13
5.4	Klasa Tile	13
6	Weryfikacja i walidacja	15
6.1	Testowanie	15
6.2	Wykryte błędy	15
7	Podsumowanie i wnioski	17
7.1	Napotkane problemy	17
7.2	Możliwy rozwój	17
	Bibliografia	19
	Spis skrótów i symboli	21
	Źródła	23
	Lista dodatkowych plików, uzupełniających tekst pracy	25
	Spis rysunków	27
	Spis tabel	29

Rozdział 1

Wstęp

Gry planszowe są formą rozrywki dla ludzi od wieków jak i formą intelektualnego wyzwania. We współczesnych czasach znaczna część życia podlega cyfryzacji, od komunikacji do płatności bankowych. Z tego powodu również wszelkie gry planszowe są przenoszone do cyfrowego świata. Takie rozwiązanie pozwala na rozpoczęcie rozgrywki ulubionej gry planszowej w świecie online wraz z graczami z całego świata bez żadnych ograniczeń.

1.1 Cel pracy

Celem pracy jest zaimplementowanie gry planszowej 2.5D w unity. Dodatkowo w ramach gry zostanie zaimplementowany gracz komputerowy z wykorzystaniem uczenia maszynowego.

1.2 Zakres pracy

- Przegląd algorytmów uczenia maszynowego dostępnego w unity.
- Zaimplementowanie graczy komputerowych z algorytmem uczenia maszynowego oraz graczy komputerowych z predefiniowanymi ruchami.
- Zaimplementowanie gry planszowej Rummikub 2.5D w unity.
- Zaprojektowanie mapy do gry.

1.3 Charakterystyka rozdziałów

- **Wstęp** - przedstawienie celu oraz zakres pracy, określenie grup docelowych
- **Analiza tematu** - zawiera ogólną analizę wszystkich możliwych rozwiązań tego projektu.

- Wymagania i narzędzia -
- Specyfikacja zewnętrzna -
- Specyfikacja wewnętrzna -
- Weryfikacja i walidacja -
- Podsumowanie i wnioski -

Rozdział 2

Analiza tematu

Tematem pracy jest cyfrowa gra planszowa z wykorzystaniem algorytmów sztucznej inteligencji. Kluczowym problemem jest zaimplementowanie inteligentnego gracza, który poprzez system prób i błędów, nauczy się grać. Wybranie gry planszowej ma duże znaczenie, im bardziej skomplikowane zasady gry, tym trudniej będzie modelowi nauczyć się zasad. Do realizacji tego projektu została wybrana gra planszowa o nazwie Rummikub. Rummikub to gra planszowa, w której gracze rywalizują, układając płytki w:

- grupy - te same liczby w różnych kolorach
- sekwencje - kolejne liczby w tym samym kolorze.

Celem graczy jest jako pierwszy wyłożyć wszystkie płytki, przy czym pierwszy ruch wymaga uzyskania układów o sumie co najmniej 30 punktów. Gra kończy się, gdy gracz wyłoży wszystkie płytki, a punkty zależą od liczby pozostałych płytek u przeciwników.

2.1 Sformułowanie problemu

Cyfrowe wersje gier planszowych często są projektowane z myślą o rozgrywkach online z innymi graczami za pośrednictwem internetu. Jednak stworzenie gry z komputerowym przeciwnikiem opartym na algorytmach nauczania maszynowego wymaga odmiennego podejścia i zastosowania innych technik programistycznych, model nie powinien się uczyć na graniu z żywymi graczami, gdyż zajęło by mu to za długo. Algorytmy AI potrzebują dużej ilości powtórzeń aby nauczyć się zasad gry. W projekcie zostanie uwzględniony jeden żywy gracz, gracze komputerowi z predefiniowanymi ruchami oraz komputerowy gracz z algorytmem nauczania maszynowego.

2.2 Znane rozwiązania

Na rynku jest wiele gier planszowych, najpopularniejsze z nich doczekały się swojej cyfrowej wersji. Są nimi między innymi:

- Monopoli
- Cantan
- Rummikub - Wydana gra online Rummikub jest nastawiona na grę sieciową do komunikacji innych graczy z całego świata. Gra jest zaimplementowana w grafice 2D bez algorytmów sztucznej inteligencji.

Rozdział 3

Wymagania i narzędzia

W tej sekcji przedstawiono wymagania funkcjonalne i нефункционаłne projektu oraz narzędzia technologiczne i programistyczne, które zostały wykorzystane do jego realizacji. Opisane zostaną także kluczowe zasoby niezbędne do wdrożenia rozwiązania, takie jak środowiska programistyczne, biblioteki, frameworki oraz sprzęt.

3.1 Wymagania funkcjonalne

Interakcja z mapą :

- Gra musi pozwolić graczowi i agentowi na wybór płytki, którą chcą postawić bądź wejść z nią w interakcje.
- Gracz oraz agent powinni być w stanie postawić nową płytkę na mapie, przesunąć istniejącą bądź usunąć ją z mapy, jeśli jest to możliwe.
- Agent oraz gracz muszą wyłożyć sekwencje bądź grupy płytek jak najszybciej.
- Gra musi ograniczać graczy, aby nie mogli tworzyć sekwencji niedozwolonych w zasadach gry.

Ruchy gracza :

- Użytkownik musi mieć możliwość widzieć swoje płytki, wykladać je na mapę oraz tworzyć sekwencje bądź grupy
- W trakcie tury gra na bieżąco weryfikuje czy daną płytkę można postawić, pod koniec tury sprawdza czy każdy wyłożony ciąg płytek posiada co najmniej 3 płytki w jednym ciągu.
- Gracz powinien móc zakończyć turę, cofnąć wszystkie akcje oraz dobrać nową płytkę.
- Użytkownik powinien móc posortować swoje płytki.

Komputerowi gracze :

- Jeśli jest to pierwsza tura gracza:
 - Sprawdza możliwość wyłożenia sekwencji, gdzie jej suma wynosi conajmniej 30 lub więcej.
 - Sprawdza możliwość wyłożenia grupy, gdzie jej suma wynosi conajmniej 30 lub więcej.
- Gdy pierwsza tura zostanie zakończona:
 - Sprawdza możliwość wyłożenia sekwencji i grup.
 - Jeśli to możliwe, dokłada płytki do innych grup i sekwencji.

Tryby gry :

- Użytkownik przeciwko trzem graczom komputerowym
- Trzej gracze komputerowi przeciwko agentowi, tryb służy głównie do nauki modelu
- Użytkownik przeciwko modelowi, tryb służy głównie do testowania, czy model jest poprawnie nauczony.
- Użytkownik przeciwko dwóm graczom komputerowym i jednemu agentowi.

3.2 Wymagania нефunkcjonalne

- **Kompatybilność :**
 - Gra musi działać na systemach Windows.
 - Rozgrywka wymaga klawiatury i myszki do sterowania.
- **Użyteczność :**
 - Interfejs użytkownika powinien być intuicyjny i zrozumiały dla nowych graczy.
 - Kolorystyka i oznaczenia płytek powinny odpowiadać klasycznym zasadom gry Rummikub.
- **Niezawodność :**
 - Gra nie zezwala graczom na wykonanie ruchów wbrew zasadom.

3.3 Opis narzędzi

W tej sekcji przedstawiono narzędzia, które zostały wykorzystane do zrealizowania projektu.

3.3.1 Unity

Środowisko gry zostało zaimplementowane z silnika Unity w najnowszej stabilnej wersji 2022.3.48f1. Unity jest wieloplatformowym silnikiem do tworzenia gier i aplikacji 3D jak i 2D stworzony przez Unity Technologies. Jest szeroko stosowany nie tylko do gier wideo, ale również do symulatorów, wizualizacji interaktywnych oraz aplikacji. Unity oferuje dużo możliwości, takie jak:

- Tworzenie modeli 2D i 3D
- Animacje i symulacja fizyki
- Renderowanie grafiki

3.3.2 C#

C# (C-Sharp) to nowoczesny, obiektowy język programowania opracowany przez firmę Microsoft. Łączy zalety języków C++ oraz Java, co czyni go uniwersalnym przy użytkowaniu. Może być wykorzystany do tworzenia gier wideo czy aplikacji mobilnych.

C# wyróżnia się następującymi cechami:

- Programowanie obiektowe.
- Zarządzanie pamięcią - posiadając funkcję garbage collector, nie trzeba martwić się o wycieki pamięci.
- Wszechstronność – umożliwia tworzenie aplikacji dla systemów Windows, macOS, Android, iOS oraz gier w środowisku Unity.

C# w projekcie został wykorzystany do zaimplementowania logiki gry, komunikacji z modelem oraz zarządzania interakcji między graczem a środowiskiem w Unity.

3.3.3 Python

Python to wszechstronny, wysokopoziomowy język programowania, głównie znany z prostoty i przejrzystości kodu. Najważniejsze cechy języka Python to:

- Bogata biblioteka standardowa – zawiera moduły do pracy z danymi, plikami, sieciami, obliczeniami naukowymi i innymi zadaniami.

- Przenośność i wszechstronność – Python działa na większości systemów operacyjnych, takich jak Windows, macOS czy Linux i jest wykorzystywany w aplikacjach desktopowych, webowych, a także w badaniach naukowych.
- Poprzez istnienie wielu zewnętrznych bibliotek, takich jak NumPy, TensorFlow, PyTorch, idealnie nadaje się do analizy danych.

Python w wersji 3.9 jest wykorzystany jako środowisko do implementacji modelu uczenia maszynowego, jest to konieczne w przypadku biblioteki ML-Agents.

3.3.4 ML-Agents

ML-Agents (Machine Learning Agents) to ,opracowany przez Unity Technologies, zestaw narzędzi do wykorzystywania technik sztucznej inteligencji i uczenia maszynowego w projektach w środowisku Unity. Narzędzie pozwala na implementację agentów, którzy mogą podejmować decyzje w zaimplementowanym środowisku.

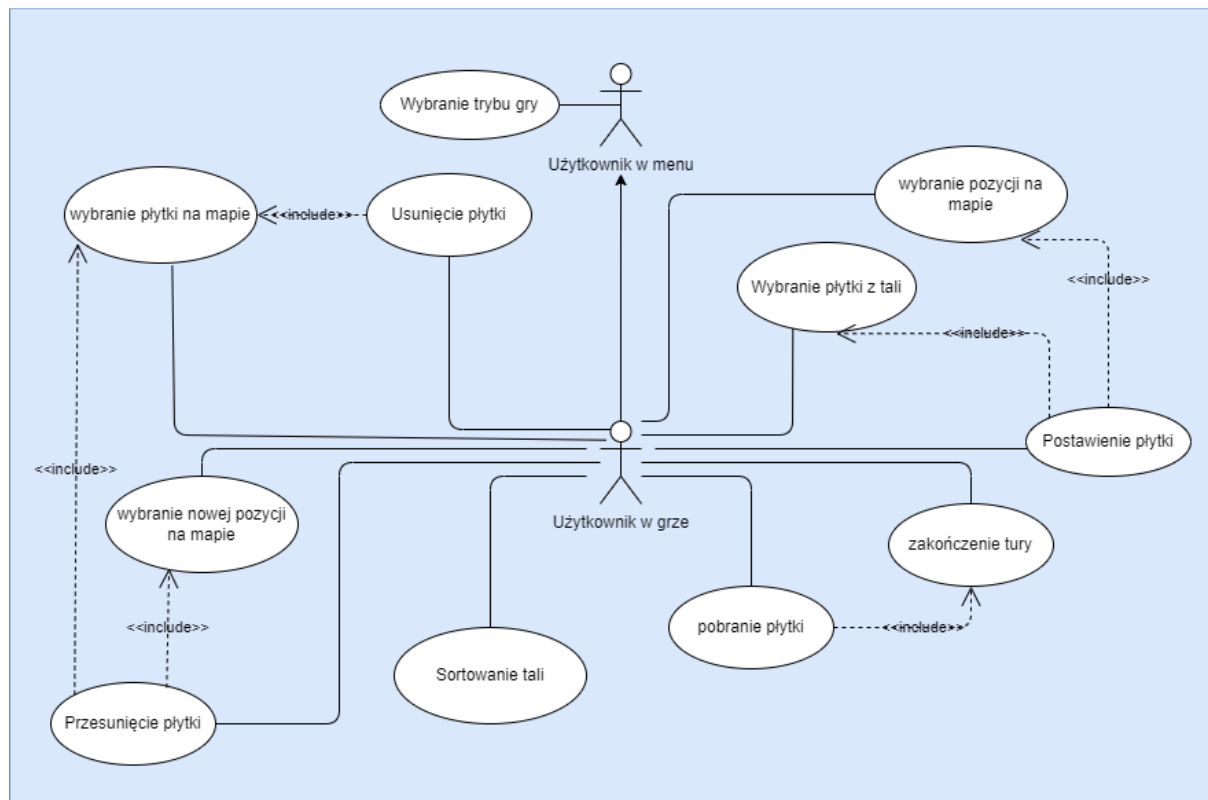
Kluczowymi cechami ML-Agents:

- Nauczanie wzmacniane - agenci uczą się na bazie dostawianych kar i nagród, jakie otrzymują.
- Narzędzie jest uniwersalne, można je zaimplementować do prostych jak i zaawansowanych gier, w których muszą podejmować decyzje.
- Agentów można zaimplementować do różnego rodzaju scenariuszy.

W tym projekcie, narzędzie, do poprawnej nauki wykorzystuje środowisko Python oraz unity. Unity jest miejscem implementacji akcji, obserwacji, nagród i kar dla modelu. Środowisko Python jest wykorzystywane do implementacji modelu oraz rozpoczynania nauki.

3.4 Przypadki użycia

Diagram przypadków użycia pokazuje możliwe funkcje dla gracza w menu oraz w grze.



Rysunek 3.1: Diagram przypadków użycia

Rozdział 4

Specyfikacja zewnętrzna

TODO

4.1 Wymagania sprzętowe i programowe

4.2 Instalacja

4.2.1 unity

4.2.2 biblioteka ml-agents

4.3 Aktywacja

4.4 Sposób obsługi

4.5 Ekran aplikacji

4.5.1 Zasady gry

4.5.2 Opis ekranu gry

Rozdział 5

Specyfikacja wewnętrzna

TODO

5.1 Biblioteka ml-agents

5.2 Klasa PlayerIA

5.3 Klasa GameController

5.4 Klasa Tile

Rozdział 6

Weryfikacja i walidacja

TODO

6.1 Testowanie

6.2 Wykryte błędy

Rozdział 7

Podsumowanie i wnioski

TODO

7.1 Napotkane problemy

7.2 Możliwy rozwój

Dodatki

Spis skrótów i symboli

Źródła

Lista dodatkowych plików, uzupełniających tekst pracy

Spis rysunków

3.1	Diagram przypadków użycia	9
-----	-------------------------------------	---

Spis tabel